

MOSCOWJS #71

Кинетический манифест

Event-driven подход к состоянию и чем его реализуют — от KRL до Effector



About

Оловянников Илья — *MoscowJS*

- Effector Community Member;
- MoscowJS организатор;
- Батя.



План доклада

План доклада



Что такое кинетический подход — концепт: события, связи, реактивность; корни в KRL

План доклада



Что такое кинетический подход — концепт: события, связи, реактивность; корни в KRL



Как это реализуют — Effector как пример + соседи: RxJS, XState, signals

План доклада



Что такое кинетический подход — концепт: события, связи, реактивность; корни в KRL



Как это реализуют — Effector как пример + соседи: RxJS, XState, signals



Чем подход выигрывает — race conditions, логика $\neq$ UI, тесты и SSR

План доклада



Что такое кинетический подход — концепт: события, связи, реактивность; корни в KRL



Как это реализуют — Effector как пример + соседи: RxJS, XState, signals



Чем подход выигрывает — race conditions, логика $\neq$ UI, тесты и SSR



Trade-offs — когда стоит, когда overkill, и какая цена

Вопрос залу 🙋

Согласны, что **веб — событийно-ориентированная среда?**



Да, конечно



Не уверен / нет

Поднимите руку — пригодится в следующие 10 минут

Данные больше не статичны

Веб 2000-х: «загрузил страницу — увидел снимок данных». Веб сегодня: данные **текут** непрерывно.

Данные больше не статичны

Веб 2000-х: «загрузил страницу — увидел снимок данных». Веб сегодня: данные **текут** непрерывно.

-  **WebSocket / SSE** — сервер пушит изменения

Данные больше не статичны

Веб 2000-х: «загрузил страницу — увидел снимок данных». Веб сегодня: данные **текут** непрерывно.

-  **WebSocket / SSE** — сервер пушит изменения
-  **Коллаборативность (Figma, доки)** — несколько источников правды сразу

Данные больше не статичны

Веб 2000-х: «загрузил страницу — увидел снимок данных». Веб сегодня: данные **текут** непрерывно.

-  **WebSocket / SSE** — сервер пушит изменения
-  **Коллаборативность** (Figma, доки) — несколько источников правды сразу
-  **Поток ввода** — печать, скролл, drag, геолокация

Данные больше не статичны

Веб 2000-х: «загрузил страницу — увидел снимок данных». Веб сегодня: данные **текут** непрерывно.

-  **WebSocket / SSE** — сервер пушит изменения
-  **Коллаборативность** (Figma, доки) — несколько источников правды сразу
-  **Поток ввода** — печать, скролл, drag, геолокация
-  **Оптимистичные апдейты, гонки запросов, отмена**

Данные больше не статичны

Веб 2000-х: «загрузил страницу — увидел снимок данных». Веб сегодня: данные **текут** непрерывно.

-  **WebSocket / SSE** — сервер пушит изменения
-  **Коллаборативность** (Figma, доки) — несколько источников правды сразу
-  **Поток ввода** — печать, скролл, drag, геолокация
-  **Оптимистичные апдейты, гонки запросов, отмена**

Если данные постоянно в движении — почему мы пишем код так, будто состояние стоит на месте и мы его «обновляем командами»?



Проблема

Почему императивный код не масштабируется

Как растёт императивный код

```
function SearchUsers() {  
  const [q, setQ] = useState('')  
  const [users, setUsers] = useState([])  
  const [loading, setLoading] = useState(false)  
  
  // добавили «живой поиск»  
  useEffect(() => {  
    if (q.length < 2) return  
    // ответы приходят вразнобой  
    api.search(q).then(setUsers)  
  }, [q])  
  
  // та же логика из формы — продублирована  
  async function onSubmit() {  
    setUsers(await api.search(q))  
  }  
}
```


Как растёт императивный код

```
function SearchUsers() {  
  const [q, setQ] = useState('')  
  const [users, setUsers] = useState([])  
  const [loading, setLoading] = useState(false)  
  
  // добавили «живой поиск»  
  useEffect(() => {  
    if (q.length < 2) return  
    // ответы приходят вразнобой  
    api.search(q).then(setUsers)  
  }, [q])  
  
  // та же логика из формы — продублирована  
  async function onSubmit() {  
    setUsers(await api.search(q))  
  }  
}
```

Как растёт императивный код

```
function SearchUsers() {  
  const [q, setQ] = useState('')  
  const [users, setUsers] = useState([])  
  const [loading, setLoading] = useState(false)  
  
  // добавили «живой поиск»  
  useEffect(() => {  
    if (q.length < 2) return  
    // ответы приходят вразнобой  
    api.search(q).then(setUsers)  
  }, [q])  
  
  // та же логика из формы — продублирована  
  async function onSubmit() {  
    setUsers(await api.search(q))  
  }  
}
```

Как растёт императивный код

```
function SearchUsers() {  
  const [q, setQ] = useState('')  
  const [users, setUsers] = useState([])  
  const [loading, setLoading] = useState(false)  
  
  // добавили «живой поиск»  
  useEffect(() => {  
    if (q.length < 2) return  
    // ответы приходят вразнобой  
    api.search(q).then(setUsers)  
  }, [q])  
  
  // та же логика из формы — продублирована  
  async function onSubmit() {  
    setUsers(await api.search(q))  
  }  
}
```

Как растёт императивный код

```
function SearchUsers() {  
  const [q, setQ] = useState('')  
  const [users, setUsers] = useState([])  
  const [loading, setLoading] = useState(false)  
  
  // добавили «живой поиск»  
  useEffect(() => {  
    if (q.length < 2) return  
    // ответы приходят вразнобой  
    api.search(q).then(setUsers)  
  }, [q])  
  
  // та же логика из формы — продублирована  
  async function onSubmit() {  
    setUsers(await api.search(q))  
  }  
}
```

-  Логика пририта к компоненту — внутри `useEffect` и обработчика

Как растёт императивный код

```
function SearchUsers() {  
  const [q, setQ] = useState('')  
  const [users, setUsers] = useState([])  
  const [loading, setLoading] = useState(false)  
  
  // добавили «живой поиск»  
  useEffect(() => {  
    if (q.length < 2) return  
    // ответы приходят вразнобой  
    api.search(q).then(setUsers)  
  }, [q])  
  
  // та же логика из формы — продублирована  
  async function onSubmit() {  
    setUsers(await api.search(q))  
  }  
}
```

-  Логика прибита к компоненту — внутри `useEffect` и обработчика
-  **Гонка:** ответы приходят не по порядку — старый затирает свежий

Как растёт императивный код

```
function SearchUsers() {  
  const [q, setQ] = useState('')  
  const [users, setUsers] = useState([])  
  const [loading, setLoading] = useState(false)  
  
  // добавили «живой поиск»  
  useEffect(() => {  
    if (q.length < 2) return  
    // ответы приходят вразнобой  
    api.search(q).then(setUsers)  
  }, [q])  
  
  // та же логика из формы — продублирована  
  async function onSubmit() {  
    setUsers(await api.search(q))  
  }  
}
```

-  Логика прибита к компоненту — внутри `useEffect` и обработчика
-  **Гонка:** ответы приходят не по порядку — старый затирает свежий
-  Второй триггер → дублирование: форма повторяет эффект

Как растёт императивный код

```
function SearchUsers() {  
  const [q, setQ] = useState('')  
  const [users, setUsers] = useState([])  
  const [loading, setLoading] = useState(false)  
  
  // добавили «живой поиск»  
  useEffect(() => {  
    if (q.length < 2) return  
    // ответы приходят вразнобой  
    api.search(q).then(setUsers)  
  }, [q])  
  
  // та же логика из формы — продублирована  
  async function onSubmit() {  
    setUsers(await api.search(q))  
  }  
}
```

-  Логика пририта к компоненту — внутри `useEffect` и обработчика
-  **Гонка:** ответы приходят не по порядку — старый затирает свежий
-  Второй триггер → дублирование: форма повторяет эффект
-  **Не протестировать** без рендера и моков таймеров

Как растёт императивный код

```
function SearchUsers() {  
  const [q, setQ] = useState('')  
  const [users, setUsers] = useState([])  
  const [loading, setLoading] = useState(false)  
  
  // добавили «живой поиск»  
  useEffect(() => {  
    if (q.length < 2) return  
    // ответы приходят вразнобой  
    api.search(q).then(setUsers)  
  }, [q])  
  
  // та же логика из формы — продублирована  
  async function onSubmit() {  
    setUsers(await api.search(q))  
  }  
}
```

-  Логика прибита к компоненту — внутри `useEffect` и обработчика
-  **Гонка:** ответы приходят не по порядку — старый затирает свежий
-  Второй триггер → дублирование: форма повторяет эффект
-  **Не протестировать** без рендера и моков таймеров

Чем больше триггеров и `async` — тем хуже. Корень: логика связана с компонентом и порядком выполнения.

До / После

❌ Императивно — логика пририта к UI

```
function SearchUsers() {  
  const [q, setQ] = useState('')  
  const [users, setUsers] = useState([])  
  
  useEffect(() => {  
    if (q.length < 2) return  
    api.search(q).then(setUsers) // гонка  
  }, [q])  
  
  async function onSubmit() {  
    setUsers(await api.search(q)) // дубль  
  }  
}
```

✅ Кинетически — граф решает порядок

```
const searchUsers = createEvent<string>()  
const $users = createStore<User[]>([])  
const searchFx = createEffect(api.search)  
  
sample({  
  clock: searchUsers,  
  filter: (q) => q.length >= 2,  
  target: searchFx,  
})  
$users.on(searchFx.doneData, (_, u) => u)
```

До / После

❌ Императивно — логика пририта к UI

```
function SearchUsers() {  
  const [q, setQ] = useState('')  
  const [users, setUsers] = useState([])  
  
  useEffect(() => {  
    if (q.length < 2) return  
    api.search(q).then(setUsers) // гонка  
  }, [q])  
  
  async function onSubmit() {  
    setUsers(await api.search(q)) // дубль  
  }  
}
```

✅ Кинетически — граф решает порядок

```
const searchUsers = createEvent<string>()  
const $users = createStore<User[]>([])  
const searchFx = createEffect(api.search)  
  
sample({  
  clock: searchUsers,  
  filter: (q) => q.length >= 2,  
  target: searchFx,  
})  
$users.on(searchFx.doneData, (_, u) => u)
```

Слева — как выполнить. Справа — когда и откуда. Гонка исчезла не магией — снимок берётся атомарно.

Два мировоззрения

Корень боли: **императив** диктует «как» (порядок шагов), а **декларатив** описывает «что» (связи). Отсюда два мировоззрения:

	Дискретное / императивное	Кинетическое / декларативное
Состояние	снимок, который ты мутируешь	поток через граф связей
Ты пишешь	порядок действий	связи: «когда X — следует Y»
Событие	повод вызвать обработчик	факт, что сам находит свои правила
Логика	размазана по компонентам	в модели, отдельно от UI
Примеры	Redux, Zustand, <code>useState</code>	Effector, RxJS, XState, signals

Два мировоззрения

Корень боли: **императив** диктует «как» (порядок шагов), а **декларатив** описывает «что» (связи). Отсюда два мировоззрения:

	Дискретное / императивное	Кинетическое / декларативное
Состояние	снимок, который ты мутируешь	поток через граф связей
Ты пишешь	порядок действий	связи: «когда X — следует Y»
Событие	повод вызвать обработчик	факт, что сам находит свои правила
Логика	размазана по компонентам	в модели, отдельно от UI
Примеры	Redux, Zustand, <code>useState</code>	Effector, RxJS, XState, signals

Дисклеймер: дискретный подход — не «плохой». Для половины задач он честнее и проще. Доклад не про «выбросьте `useState`», а про то, **когда и почему** второй взгляд мощнее. К **trade-offs** вернёмся.



Часть 1

Что такое кинетический подход

Что такое «кинетика»?

Греч. *kinētikos* — «относящийся к движению» (κίνησις — движение). В классической механике движение описывают три раздела:

Статика

равновесие тел — силы **без** движения

Кинематика

геометрия движения (траектория, скорость) — **без причин**

Динамика

движение **с учётом сил и масс** (законы Ньютона)

Строго говоря, «кинетики» как раздела механики в русской традиции нет. Термин живёт в другом:

- **Физическая кинетика** — статфизика: неравновесные процессы и перенос (диффузия, теплопроводность), уравнение Больцмана
- **Химическая кинетика** — скорости реакций и их механизмы
- **Англ. *kinetics*** — часть динамики о связи сил и движения ($\approx$ рус. «динамика») → вероятный источник «kinetic» в KRL

Что такое «кинетика»?

Греч. *kinētikos* — «относящийся к движению» (κίνησις — движение). В классической механике движение описывают три раздела:

Статика

равновесие тел — силы **без** движения

Кинематика

геометрия движения (траектория, скорость) — **без причин**

Динамика

движение **с учётом сил и масс** (законы Ньютона)

Строго говоря, «кинетики» как раздела механики в русской традиции нет. Термин живёт в другом:

- **Физическая кинетика** — статфизика: неравновесные процессы и перенос (диффузия, теплопроводность), уравнение Больцмана
- **Химическая кинетика** — скорости реакций и их механизмы
- **Англ. *kinetics*** — часть динамики о связи сил и движения ($\approx$ рус. «динамика») → вероятный источник «kinetic» в KRL



Общий знаменатель: кинетика — про скорость и эволюцию процессов во времени, а не про статичное состояние.

Кинетика как метафора

В стейт-менеджменте «кинетика» — не из учебника физики (это **имя из KRL**), а **метафора**. Берём её за то, что в ней ценно:

Физическая интуиция

В управлении состоянием

Кинетика как метафора

В стейт-менеджменте «кинетика» — не из учебника физики (это **имя из KRL**), а **метафора**. Берём её за то, что в ней ценно:

Физическая интуиция

В управлении состоянием

Импульс (толкнули)

Event — клик, ввод, сообщение из сокета

Кинетика как метафора

В стейт-менеджменте «кинетика» — не из учебника физики (это **имя из KRL**), а **метафора**. Берём её за то, что в ней ценно:

Физическая интуиция

В управлении состоянием

Импульс (толкнули)

Event — клик, ввод, сообщение из сокета

Траектория

путь данных через граф правил

Кинетика как метафора

В стейт-менеджменте «кинетика» — не из учебника физики (это **имя из KRL**), а **метафора**. Берём её за то, что в ней ценно:

Физическая интуиция

В управлении состоянием

Импульс (толкнули)

Event — клик, ввод, сообщение из сокета

Траектория

путь данных через граф правил

Препятствие

условие / фильтр — пропустить или нет

Кинетика как метафора

В стейт-менеджменте «кинетика» — не из учебника физики (это имя из **KRL**), а **метафора**. Берём её за то, что в ней ценно:

Физическая интуиция	В управлении состоянием
Импульс (толкнули)	Event — клик, ввод, сообщение из сокета
Траектория	путь данных через граф правил
Препятствие	условие / фильтр — пропустить или нет
Результат движения	эффект: запрос, запись, уведомление

Кинетика как метафора

В стейт-менеджменте «кинетика» — не из учебника физики (это имя из **KRL**), а **метафора**. Берём её за то, что в ней ценно:

Физическая интуиция	В управлении состоянием
Импульс (толкнули)	Event — клик, ввод, сообщение из сокета
Траектория	путь данных через граф правил
Препятствие	условие / фильтр — пропустить или нет
Результат движения	эффект: запрос, запись, уведомление

Состояние — не «точка», а **траектория** в графе.

У данных есть динамика – откуда они пришли, какими стали и куда идут.

KRL — язык для «Живого Веба»

Kinetic Rule Language

Что это

- Правило-ориентированный язык, **2007–2008**
- Программа = набор **правил** (rulesets), реагирующих на события
- Часть движка **KRE** (Kinetic Rules Engine), open-source

Кто создал

- Фил Виндли (Phil Windley), компания **Kynetx**
- Эпоха Twitter / iPhone / открытых API — веб стал реактивным

Главная идея

Веб — это не страницы, это **потоки событий**.

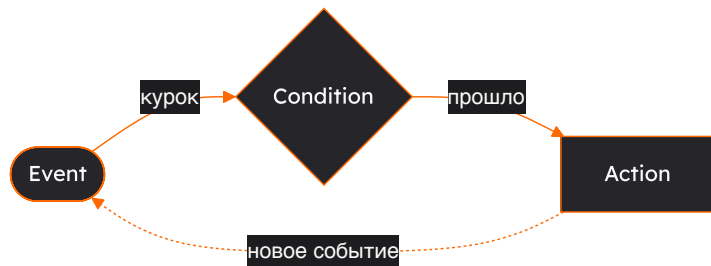
Каждый пользователь — «**personal cloud**»: облако правил, реагирующих на события его жизни. Три кита:

-  **Entity orientation** — правила работают *от имени сущности* (юзер, устройство)
-  **Event binding** — связывают *паттерны событий* с *действиями*
-  **Persistent values** — у правил своя «память» без БД

ECA — сердце KRL

Каждое правило KRL устроено как **ECA (Event-Condition-Action)**:

```
rule good_morning {  
  select when pageview url #example.com# // EVENT: что случилось  
  if morning() then                       // CONDITION: проверка  
  notify("Welcome!", "Good morning!")    // ACTION: реакция  
}
```



Вот он, наш первый граф.

Узлы — событие, условие, действие;

рёбра — что за чем следует.

Правило — путь **Event** → **Condition** → **Action**.

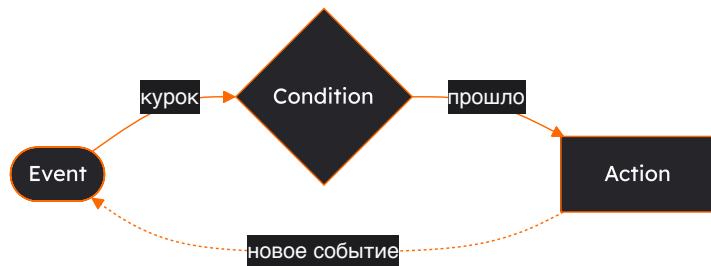
Не прошло условие — правило молчит.

А действие рождает новое событие →
правила сцепляются в **граф**.

ECA — сердце KRL

Каждое правило KRL устроено как **ECA (Event-Condition-Action)**:

```
rule good_morning {  
  select when pageview url #example.com# // EVENT: что случилось  
  if morning() then                       // CONDITION: проверка  
  notify("Welcome!", "Good morning!")    // ACTION: реакция  
}
```



Вот он, наш первый граф.

Узлы — событие, условие, действие;

рёбра — что за чем следует.

Правило — путь **Event** → **Condition** → **Action**.

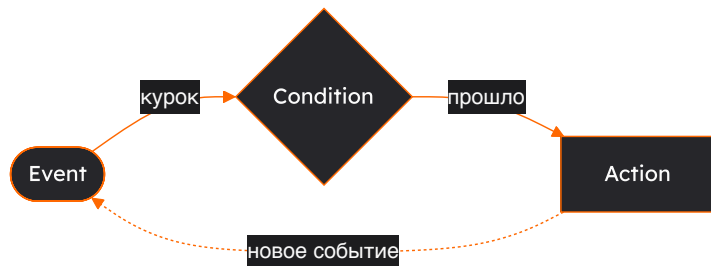
Не прошло условие — правило молчит.

А действие рождает новое событие →
правила сцепляются в **граф**.

ECA — сердце KRL

Каждое правило KRL устроено как **ECA (Event-Condition-Action)**:

```
rule good_morning {  
  select when pageview url #example.com# // EVENT: что случилось  
  if morning() then                       // CONDITION: проверка  
    notify("Welcome!", "Good morning!") // ACTION: реакция  
}
```



Вот он, наш первый граф.

Узлы — событие, условие, действие;

рёбра — что за чем следует.

Правило — путь **Event → Condition → Action**.

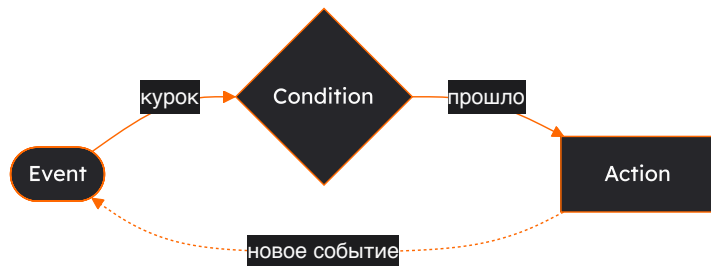
Не прошло условие — правило молчит.

А действие рождает новое событие →
правила сцепляются в **граф**.

ECA — сердце KRL

Каждое правило KRL устроено как **ECA (Event-Condition-Action)**:

```
rule good_morning {  
  select when pageview url #example.com# // EVENT: что случилось  
  if morning() then                       // CONDITION: проверка  
  notify("Welcome!", "Good morning!")    // ACTION: реакция  
}
```



Вот он, наш первый граф.

Узлы — событие, условие, действие;

рёбра — что за чем следует.

Правило — путь **Event** → **Condition** → **Action**.

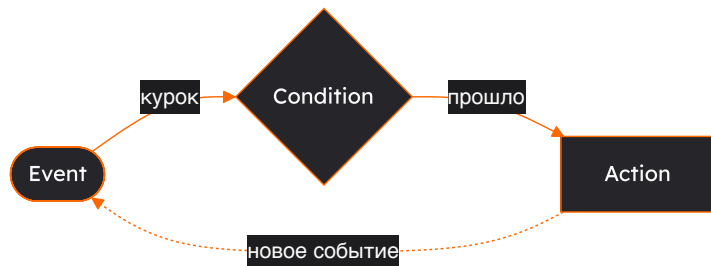
Не прошло условие — правило молчит.

А действие рождает новое событие →
правила сцепляются в **граф**.

ECA — сердце KRL

Каждое правило KRL устроено как **ECA (Event-Condition-Action)**:

```
rule good_morning {  
  select when pageview url #example.com# // EVENT: что случилось  
  if morning() then                       // CONDITION: проверка  
  notify("Welcome!", "Good morning!")    // ACTION: реакция  
}
```



Вот он, наш первый граф.

Узлы — событие, условие, действие;

рёбра — что за чем следует.

Правило — путь **Event** → **Condition** → **Action**.

Не прошло условие — правило молчит.

А действие рождает новое событие →
правила сцепляются в **граф**.

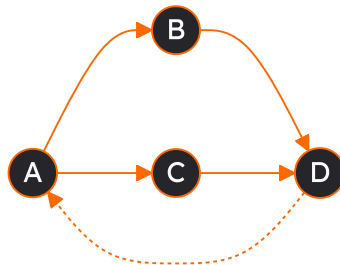
Тот же ECA лежит под Effector (`event` → `filter` → `target`), под reducer'ами, под стейт-машинами. Подходу почти 20 лет.

Что такое граф?

Мы уже пару раз сказали «граф». Это всего две вещи:

-  **Узлы** (вершины) — «штуки»
-  **Рёбра** — связи между ними

Есть направление (стрелки) → граф **направленный**: связь идёт в одну сторону. Ребро может вести и назад — это **обратная связь** (цикл).



4 узла · рёбра со стрелками · пунктир — обратная связь

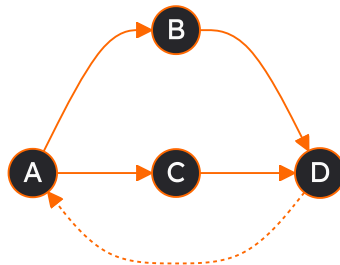
Что такое граф?

Мы уже пару раз сказали «граф». Это всего две вещи:

-  **Узлы** (вершины) — «штуки»
-  **Рёбра** — связи между ними

Есть направление (стрелки) → граф **направленный**: связь идёт в одну сторону. Ребро может вести и назад — это **обратная связь** (цикл).

У нас: узлы — **события, состояния, эффекты**; рёбра — **связи**. Данные текут по стрелкам — мы лишь задаём русло.



4 узла · рёбра со стрелками · пунктир — обратная связь

Три столпа подхода

Три столпа подхода

1. Event-driven

Уведомление, а не команда.

Императив: «Сделай!»

Вопрос: «Сделаешь ли?»

Event: «Это случилось».

Событие не знает, кто отреагирует

→ слабая связанность.

Три столпа подхода

1. Event-driven

Уведомление, а не команда.

Императив: «Сделай!»

Вопрос: «Сделаешь ли?»

Event: «Это случилось».

Событие не знает, кто отреагирует
→ слабая связанность.

2. Declarative

Связи вместо порядка.

Ты не пишешь «сначала А, потом Б».

Ты объявляешь: «Б зависит от А».

Порядок вычислит граф.

Три столпа подхода

1. Event-driven

Уведомление, а не команда.

Императив: «Сделай!»

Вопрос: «Сделаешь ли?»

Event: «Это случилось».

Событие не знает, кто отреагирует
→ слабая связанность.

2. Declarative

Связи вместо порядка.

Ты не пишешь «сначала А, потом Б».

Ты объявляешь: «Б зависит от А».

Порядок вычислит граф.

3. Reactivity

Граф пересчитывается сам.

Изменился источник →
производные обновились
автоматически,
в один тик,
без ручных вызовов.

Три столпа подхода

1. Event-driven

Уведомление, а не команда.

Императив: «Сделай!»

Вопрос: «Сделаешь ли?»

Event: «Это случилось».

Событие не знает, кто отреагирует
→ слабая связанность.

2. Declarative

Связи вместо порядка.

Ты не пишешь «сначала А, потом Б».

Ты объявляешь: «Б зависит от А».

Порядок вычислит граф.

3. Reactivity

Граф пересчитывается сам.

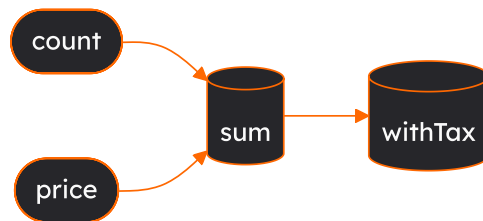
Изменился источник →
производные обновились
автоматически,
в один тик,
без ручных вызовов.

Эти три вещи делают подход «кинетическим» — **независимо от библиотеки.**

Что такое реактивность?

Меняешь **источник** — всё зависимое **пересчитывается** само.

Императивно: поменял `count` — не забудь дёрнуть `updateSum()`, `updateTax()` ... забыл — баг.



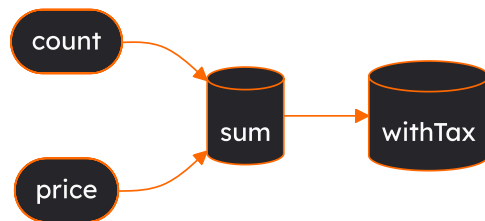
меняем **count** → **sum** пересчитался → **withTax** пересчитался

Что такое реактивность?

Меняешь **источник** — всё зависимое **пересчитывается** само.

Императивно: поменял `count` — не забудь дёрнуть `updateSum()`, `updateTax()` ... забыл — баг.

Реактивно: один раз объявил `sum = count × price` — дальше оно само.



меняем **count** → **sum** пересчитался → **withTax** пересчитался

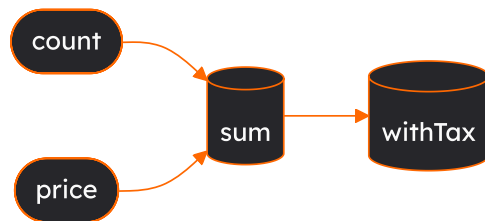
Что такое реактивность?

Меняешь **источник** — всё зависимое **пересчитывается** само.

Императивно: поменял `count` — не забудь дёрнуть `updateSum()`, `updateTax()` ... забыл — баг.

Реактивно: один раз объявил `sum = count × price` — дальше оно само.

Это тот же граф: меняется узел — изменение бежит по рёбрам к зависимым.



меняем **count** → **sum** пересчитался → **withTax** пересчитался

Реактивность — не изобретение фронтенда

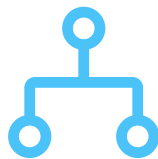
Идея «данные движутся сами» старше React лет на тридцать.

Год	Технология	Реактивность
1979	VisiCalc / Excel	<code>=A1+B1</code> пересчитывается при изменении ячеек
1995	Delphi / VB	data-binding: UI привязан к переменным
2010	Knockout / Ember	observable, уведомления об изменениях
2013	React	перерисовка дерева при смене стейта
2015	MobX	прозрачная реактивность («магия»)
2018	Effector	реактивность через явный граф
2020+	Solid / Signals	fine-grained: обновляется только нужный узел

Реактивность — не изобретение фронтенда

Идея «данные движутся сами» старше React лет на тридцать.

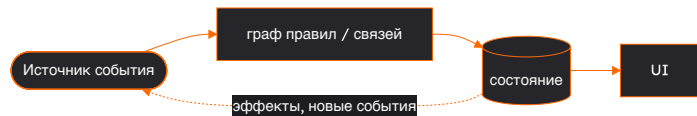
Год	Технология	Реактивность
1979	VisiCalc / Excel	<code>=A1+B1</code> пересчитывается при изменении ячеек
1995	Delphi / VB	data-binding: UI привязан к переменным
2010	Knockout / Ember	observable, уведомления об изменениях
2013	React	перерисовка дерева при смене стейта
2015	MobX	прозрачная реактивность («магия»)
2018	Effector	реактивность через явный граф
2020+	Solid / Signals	fine-grained: обновляется только нужный узел



Часть 2

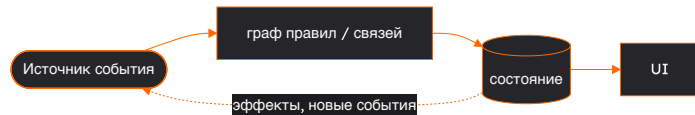
Как это реализуют — карта примитивов

Общая модель любой реализации



Роль	Effector	RxJS	Redux	Signals
Событие / намерение	event	Subject	action	вызов сеттера
Состояние	store (\$)	BehaviorSubject	reducer state	signal
Производное	combine / .map	combineLatest	selector	computed
Связь / правило	sample	операторы pipe	middleware / saga	автозависимость
Сайд-эффект	effect	оператор + subscribe	thunk / saga	effect

Общая модель любой реализации



Роль	Effector	RxJS	Redux	Signals
Событие / намерение	<code>event</code>	<code>Subject</code>	<code>action</code>	вызов сеттера
Состояние	<code>store (\$)</code>	<code>BehaviorSubject</code>	<code>reducer</code> <code>state</code>	<code>signal</code>
Производное	<code>combine / .map</code>	<code>combineLatest</code>	<code>selector</code>	<code>computed</code>
Связь / правило	<code>sample</code>	операторы <code>pipe</code>	<code>middleware</code> / <code>saga</code>	автозависимость
Сайд-эффект	<code>effect</code>	оператор + <code>subscribe</code>	<code>thunk</code> / <code>saga</code>	<code>effect</code>

Примитивы разные — подход один. Дальше код на **Effector**, но мысль переносится на любую строку.

Effector — явный граф

Все связи записаны в коде — и складываются в граф:

```
const todoAdded = createEvent<string>() // событие
const $todos = createStore<Todo[]>([]) // состояние
const saveFx = createEffect(saveTodos) // эффект

sample({                                // правило (ECA)
  clock: todoAdded,                    // когда
  source: $todos,                      // взять
  filter: Boolean,                     // если
  fn: addTodo,                        // преобразовать
  target: $todos,                     // положить
})

// автосохранение
sample({ clock: $todos, target: saveFx })
```



Effector — явный граф

Все связи записаны в коде — и складываются в граф:

```
const todoAdded = createEvent<string>() // событие
const $todos = createStore<Todo[]>([]) // состояние
const saveFx = createEffect(saveTodos) // эффект

sample({
  // правило (ECA)
  clock: todoAdded, // когда
  source: $todos, // взять
  filter: Boolean, // если
  fn: addTodo, // преобразовать
  target: $todos, // положить
})

// автосохранение
sample({ clock: $todos, target: saveFx })
```



Effector — явный граф

Все связи записаны в коде — и складываются в граф:

```
const todoAdded = createEvent<string>() // событие
const $todos = createStore<Todo[]>([]) // состояние
const saveFx = createEffect(saveTodos) // эффект

sample({                                // правило (ECA)
  clock: todoAdded,                    // когда
  source: $todos,                      // взять
  filter: Boolean,                     // если
  fn: addTodo,                        // преобразовать
  target: $todos,                     // положить
})

// автосохранение
sample({ clock: $todos, target: saveFx })
```



Effector — явный граф

Все связи записаны в коде — и складываются в граф:

```
const todoAdded = createEvent<string>() // событие
const $todos = createStore<Todo[]>([]) // состояние
const saveFx = createEffect(saveTodos) // эффект

sample({
  // правило (ECA)
  clock: todoAdded, // когда
  source: $todos, // взять
  filter: Boolean, // если
  fn: addTodo, // преобразовать
  target: $todos, // положить
})

// автосохранение
sample({ clock: $todos, target: saveFx })
```



Effector — явный граф

Все связи записаны в коде — и складываются в граф:

```
const todoAdded = createEvent<string>() // событие
const $todos = createStore<Todo[]>([]) // состояние
const saveFx = createEffect(saveTodos) // эффект

sample({
  // правило (ECA)
  clock: todoAdded, // когда
  source: $todos, // взять
  filter: Boolean, // если
  fn: addTodo, // преобразовать
  target: $todos, // положить
})

// автосохранение
sample({ clock: $todos, target: saveFx })
```



Код слева строит этот граф: юниты — **узлы**, `sample` — **рёбра**.

Данные бегут по рёбрам, а не вызываются вручную.

Соседи по подходу

Кинетика $\neq$ Effector. Тот же подход, разные акценты — каждая делает явным **своё**:

Соседи по подходу

Кинетика $\neq$ Effector. Тот же подход, разные акценты — каждая делает явным своё:

 **RxJS** — явные потоки.

Мощная асинхронная композиция; кривая обучения крутая.

Соседи по подходу

Кинетика $\neq$ Effector. Тот же подход, разные акценты — каждая делает явным своё:

 **RxJS** — явные потоки.

Мощная асинхронная композиция; кривая обучения крутая.

 **XState** — явные режимы.

Когда важно «в каком состоянии находится фича».

Соседи по подходу

Кинетика $\neq$ Effector. Тот же подход, разные акценты — каждая делает явным своё:

 **RxJS** — явные потоки.

Мощная асинхронная композиция; кривая обучения крутая.

 **XState** — явные режимы.

Когда важно «в каком состоянии находится фича».

 **MobX / Solid / Signals** — fine-grained.

Минимум кода, «магия» автозависимостей.

Соседи по подходу

Кинетика $\neq$ Effector. Тот же подход, разные акценты — каждая делает явным своё:

 **RxJS** — явные потоки.

Мощная асинхронная композиция; кривая обучения крутая.

 **XState** — явные режимы.

Когда важно «в каком состоянии находится фича».

 **MobX / Solid / Signals** — fine-grained.

Минимум кода, «магия» автозависимостей.

 **Redux-Saga / -Observable** — слой эффектов.

Кинетика поверх дискретного Redux.

Соседи по подходу

Кинетика $\neq$ Effector. Тот же подход, разные акценты — каждая делает явным своё:

~ **RxJS** — явные потоки.

Мощная асинхронная композиция; кривая обучения крутая.

⌘ **XState** — явные режимы.

Когда важно «в каком состоянии находится фича».

⚡ **MobX / Solid / Signals** — fine-grained.

Минимум кода, «магия» автозависимостей.

≡ **Redux-Saga / -Observable** — слой эффектов.

Кинетика поверх дискретного Redux.

Выбор библиотеки = выбор того, что сделать явным: граф / потоки / режимы / вычисления.



Часть 3

Чем подход реально выигрывает

Выигрыш 1 — смерть race conditions

Императивно: между `await` состояние «уезжает»

```
async function checkout() {  
  const cart = await getCart() // ждём...  
  const user = globalUser      // а если сменился?  
  api.buy(cart, user)          // что сейчас — ???  
}
```

Кинетически: значения берутся в один тик

```
sample({  
  clock: checkoutPressed,  
  source: { cart: $cart, user: $user },  
  filter: $isAuthorized,  
  target: buyFx,  
})
```

Выигрыш 1 — смерть race conditions

Императивно: между `await` состояние «уезжает»

```
async function checkout() {  
  const cart = await getCart() // ждём...  
  const user = globalUser      // а если сменился?  
  api.buy(cart, user)          // что сейчас — ???  
}
```

Кинетически: значения берутся в один тик

```
sample({  
  clock: checkoutPressed,  
  source: { cart: $cart, user: $user },  
  filter: $isAuthorized,  
  target: buyFx,  
})
```


Выигрыш 1 — смерть race conditions

Императивно: между `await` состояние «уезжает»

```
async function checkout() {  
  const cart = await getCart() // ждём...  
  const user = globalUser      // а если сменился?  
  api.buy(cart, user)          // что сейчас — ???  
}
```

Кинетически: значения берутся в один тик

```
sample({  
  clock: checkoutPressed,  
  source: { cart: $cart, user: $user },  
  filter: $isAuthorized,  
  target: buyFx,  
})
```

Выигрыш 1 — смерть race conditions

Императивно: между `await` состояние «уезжает»

```
async function checkout() {  
  const cart = await getCart() // ждём...  
  const user = globalUser      // а если сменился?  
  api.buy(cart, user)          // что сейчас — ???  
}
```

Кинетически: значения берутся в один тик

```
sample({  
  clock: checkoutPressed,  
  source: { cart: $cart, user: $user },  
  filter: $isAuthorized,  
  target: buyFx,  
})
```

Выигрыш 1 — смерть race conditions

Императивно: между `await` состояние «уезжает»

```
async function checkout() {  
  const cart = await getCart() // ждём...  
  const user = globalUser      // а если сменился?  
  api.buy(cart, user)          // что сейчас — ???  
}
```

Кинетически: значения берутся в один тик

```
sample({  
  clock: checkoutPressed,  
  source: { cart: $cart, user: $user },  
  filter: $isAuthorized,  
  target: buyFx,  
})
```

Выигрыш 1 — смерть race conditions

Императивно: между `await` состояние «уезжает»

```
async function checkout() {  
  const cart = await getCart() // ждём...  
  const user = globalUser      // а если сменился?  
  api.buy(cart, user)          // что сейчас — ???  
}
```

Кинетически: значения берутся в один тик

```
sample({  
  clock: checkoutPressed,  
  source: { cart: $cart, user: $user },  
  filter: $isAuthorized,  
  target: buyFx,  
})
```

Выигрыш 1 — смерть race conditions

Императивно: между `await` состояние «уезжает»

```
async function checkout() {  
  const cart = await getCart() // ждём...  
  const user = globalUser      // а если сменился?  
  api.buy(cart, user)          // что сейчас — ???  
}
```

Кинетически: значения берутся в один тик

```
sample({  
  clock: checkoutPressed,  
  source: { cart: $cart, user: $user },  
  filter: $isAuthorized,  
  target: buyFx,  
})
```

Нет зазора для «дрейфа состояния» — снимок берётся атомарно. То же в RxJS через `withLatestFrom`: это свойство **подхода**, а не одной библиотеки.

Выигрыш 2 — логика отделена от UI

UI

Только генерирует события
и отображает состояние.

Не знает про бизнес-логику и сайд-эффекты.

Модель

события → связи → состояние → эффекты.

Живёт без React/Vue.

Выигрыш 2 — логика отделена от UI

UI

Только генерирует события
и отображает состояние.

Не знает про бизнес-логику и сайд-эффекты.

Модель

события → связи → состояние → эффекты.

Живёт без React/Vue.

- Модель тестируется без рендера

Выигрыш 2 — логика отделена от UI

UI

Только генерирует события
и отображает состояние.

Не знает про бизнес-логику и сайд-эффекты.

Модель

события → связи → состояние → эффекты.

Живёт без React/Vue.

- Модель тестируется без рендера
- Переносится между фреймворками

Выигрыш 2 — логика отделена от UI

UI

Только генерирует события
и отображает состояние.

Не знает про бизнес-логику и сайд-эффекты.

Модель

события → связи → состояние → эффекты.

Живёт без React/Vue.

- Модель тестируется без рендера
- Переносится между фреймворками
- Компонент становится **тонким**

Выигрыш 2 — логика отделена от UI

UI

Только генерирует события
и отображает состояние.

Не знает про бизнес-логику и сайд-эффекты.

Модель

события → связи → состояние → эффекты.

Живёт без React/Vue.

- Модель тестируется без рендера
- Переносится между фреймворками
- Компонент становится **тонким**

```
// весь компонент:  
const onClick = useUnit(todoAdded)  
<button onClick={() => onClick(text)}>  
  Добавить  
</button>
```

Выигрыш 3 — изоляция для SSR и тестов

Глобальное состояние ломается на сервере: один процесс — много запросов. Кинетические библиотеки дают изолированный экземпляр графа. В Effector — `fork()` :

```
// тест без мока React: дёрнули событие — проверили состояние
const scope = fork()
await allSettled(todoAdded, { scope, params: 'Test' })
expect(scope.getState($todos)).toHaveLength(1)
```

Выигрыш 3 — изоляция для SSR и тестов

Глобальное состояние ломается на сервере: один процесс — много запросов. Кинетические библиотеки дают изолированный экземпляр графа. В Effector — `fork()` :

```
// тест без мока React: дёрнули событие — проверили состояние
const scope = fork()
await allSettled(todoAdded, { scope, params: 'Test' })
expect(scope.getState($todos)).toHaveLength(1)
```

Выигрыш 3 — изоляция для SSR и тестов

Глобальное состояние ломается на сервере: один процесс — много запросов. Кинетические библиотеки дают изолированный экземпляр графа. В Effector — `fork()` :

```
// тест без мока React: дёрнули событие — проверили состояние
const scope = fork()
await allSettled(todoAdded, { scope, params: 'Test' })
expect(scope.getState($todos)).toHaveLength(1)
```

Выигрыш 3 — изоляция для SSR и тестов

Глобальное состояние ломается на сервере: один процесс — много запросов. Кинетические библиотеки дают изолированный экземпляр графа. В Effector — `fork()` :

```
// тест без мока React: дёрнули событие — проверили состояние
const scope = fork()
await allSettled(todoAdded, { scope, params: 'Test' })
expect(scope.getState($todos)).toHaveLength(1)
```

Выигрыш 3 — изоляция для SSR и тестов

Глобальное состояние ломается на сервере: один процесс — много запросов. Кинетические библиотеки дают изолированный экземпляр графа. В Effector — `fork()` :

```
// тест без мока React: дёрнули событие — проверили состояние
const scope = fork()
await allSettled(todoAdded, { scope, params: 'Test' })
expect(scope.getState($todos)).toHaveLength(1)
```

Тот же механизм работает на сервере: **скоуп на каждый запрос** → нет утечки состояния между пользователями.



Часть 4

Trade-offs: когда стоит и когда НЕ стоит

Самая честная часть. Без неё доклад — реклама.



Когда окупается



Когда overkill



Когда окупается

- Много асинхронности и гонок —
параллельные запросы, отмена,
оптимистичные апдейты



Когда overkill



Когда окупается

- Много асинхронности и гонок —
параллельные запросы, отмена,
оптимистичные апдейты
- **Real-time** / потоки — сокеты,
коллаборативность, живые данные



Когда overkill



Когда окупается

- Много асинхронности и гонок —
параллельные запросы, отмена,
оптимистичные апдейты
- **Real-time** / потоки — сокеты,
коллаборативность, живые данные
- **Сложные зависимости** между кусками
состояния



Когда overkill



Когда окупается

- Много асинхронности и гонок —
параллельные запросы, отмена,
оптимистичные апдейты
- **Real-time** / потоки — сокеты,
коллаборативность, живые данные
- **Сложные зависимости** между кусками
состояния
- **Логику нужно тестировать** отдельно от UI



Когда overkill



Когда окупается

- Много асинхронности и гонок — параллельные запросы, отмена, оптимистичные апдейты
- **Real-time** / потоки — сокеты, коллаборативность, живые данные
- Сложные зависимости между кусками состояния
- Логику нужно тестировать отдельно от UI
- Большая команда / долгий проект — явные связи дешевле в поддержке



Когда overkill



Когда окупается

- Много асинхронности и гонок — параллельные запросы, отмена, оптимистичные апдейты
- **Real-time / потоки** — сокеты, коллаборативность, живые данные
- **Сложные зависимости** между кусками состояния
- **Логику** нужно тестировать отдельно от UI
- **Большая команда / долгий проект** — явные связи дешевле в поддержке



Когда overkill

- **Маленькое приложение / локальный стейт** — `useState` честнее
- **Прототип / MVP** — скорость важнее архитектуры
- **Состояние в основном серверное** — часто хватает TanStack Query



Когда окупается

- Много асинхронности и гонок — параллельные запросы, отмена, оптимистичные апдейты
- **Real-time / потоки** — сокеты, коллаборативность, живые данные
- **Сложные зависимости** между кусками состояния
- **Логику нужно тестировать** отдельно от UI
- **Большая команда / долгий проект** — явные связи дешевле в поддержке



Когда overkill

- **Маленькое приложение / локальный стейт** — `useState` честнее
- **Прототип / MVP** — скорость важнее архитектуры
- **Состояние в основном серверное** — часто хватает TanStack Query
- **Команда не готова к смене мышления** — кривая обучения реальна



Когда окупается

- Много асинхронности и гонок — параллельные запросы, отмена, оптимистичные апдейты
- **Real-time / потоки** — сокет, коллаборативность, живые данные
- **Сложные зависимости** между кусками состояния
- **Логику** нужно тестировать отдельно от UI
- **Большая команда / долгий проект** — явные связи дешевле в поддержке



Когда overkill

- Маленькое приложение / локальный стейт — `useState` честнее
- **Прототип / MVP** — скорость важнее архитектуры
- **Состояние в основном серверное** — часто хватает TanStack Query
- **Команда не готова к смене мышления** — кривая обучения реальна

Это инвестиция в сложность данных. Нет сложности — нет окупаемости.

Цена, о которой молчат

Цена, о которой молчат



Дебаг графа

«Откуда прилетело это обновление?» Изменение приходит издалека — без devtools трассировать больно.

event → ? → ? → 🌟 store

Цена, о которой молчат



Дебаг графа

«Откуда прилетело это обновление?» Изменение приходит издалека — без devtools трассировать больно.

event → ? → ? → 🌟 store



Порог входа

Нужно перестроить мышление: с «команд» на **граф событий и связей**.

императив → 🧠 → граф

Цена, о которой молчат



Дебаг графа

«Откуда прилетело это обновление?» Изменение приходит издалека — без devtools трассировать больно.

event → ? → ? → 🌟 store



Порог входа

Нужно перестроить мышление: с «команд» на **граф событий и связей**.

императив → 🧠 → граф



Линейность

Императив читается **сверху вниз** — граф так не прочитать.

1 → 2 → 3 vs $\sqcap$ graph $\sqcap$

Помните вопрос в начале?

«Веб — **событийно-ориентированная** среда?»»

Помните вопрос в начале?

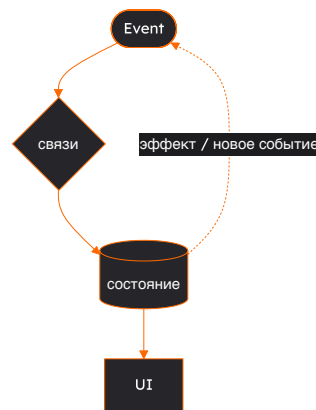
«Веб — **событийно-ориентированная** среда?»

Если да — то и состояние логичнее вести **событиями**, а не командами.
Кинетический подход просто доводит это «да» до конца.

Спасибо! 🙏

Что унести с собой:

1. Состояние **движется**, а не лежит.
2. Подход = **три столпа**: события + связи + реактивность.
3. Паттерн, а не библиотека (ECA из KRL → Effector, RxJS, XState, signals).
4. Реализации различаются тем, **что делают явным**.
5. Выигрыши: нет *race conditions*, логика $\neq$ UI, тесты и SSR.
6. **Честно**: окупается на сложности; *overkill* на простом.



три столпа в одном цикле

KRL Effector RxJS XState

- Telegram [@olovyannikov_frontend]
- GitHub [/Olovyannikov]